Fewest GPUs That Meet the SLO

Binary search on the answer, run on a real production trace

Tracks: Search (binary search on the answer, doubling search, monotone checks) **Code:** `src/FewestGpus/`. **Tests:** 78, run with `dotnet test`. **Demos:** `demo/FewestGpus.Demo` (the search) and `demo/FewestGpus.Experiments` (what-if runs), on the Azure LLM inference trace

---

0. The problem

> An LLM endpoint has a latency target on p99 time to first token. Every machine costs money. What is the fewest machines that still meets the target for this traffic?

You can't answer it by dividing traffic by one machine's capacity, because requests queue behind each other.

What the simulator here sizes: the machines that do the prompt work (prefill) up to the first token. It has no decode, so a machine is free again after the first token, like the prompt pool in Splitwise. Its one queue sends each request to the first free machine, a perfect router, which pushes the answer down. It has no batching, which pushes it up, because batching raises a real machine's prompt throughput. So the answer is an estimate with biases both ways, not a bound. The only way to know if n machines is enough is to run a check (a load test or a simulation). Checks are expensive, so the goal is to run as few as possible.

▼ Hint 1
If 12 machines meet the target, what do you know about 13?

▼ Hint 2
Write the result for n = 1, 2, 3 ... as a row of N and Y. What shape is it?

▼ Hint 3
You don't know a good upper bound. How fast can you find one?

---

1. Background

- An SLO is "a target value or range of values for a service level that is measured by an SLI", and latency targets use percentiles because a high one "shows you a plausible worst-case value" (Google SRE book, ch. 4).
- Time to first token (TTFT) is "the time from query submission to the first received token", and it "generally includes request queuing time, prefill time, and network latency" (NVIDIA NIM docs).
- Benchmark tools search the same way: Triton's Perf Analyzer has a `--binary-search` option over concurrency or request rate, stopped by a `--latency-threshold` (Perf Analyzer CLI).

---

2. The mental model

```
n:      1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16
meets:  N  N  N  N  N  N  N  N  N  N   Y   Y   Y   Y   Y   Y
```

More machines never make it slower, so the row is always N's, then Y's. The answer is the first Y.

So one check settles a whole side: an N at 6 means 1 to 5 are N, a Y at 12 means 13 to 16 are Y. Check the middle, drop the settled half, repeat. Here: 8 N, 12 Y, 10 N, 11 Y. Four checks.

You aren't searching an array. You're searching the range of possible answers, and each "element" is a check you have to run.

---

3. Three ways to search

Version	Checks for answer A	Needs an upper bound H?
Try 1, 2, 3 ...	A	No
Binary search over 1 to H	about log2(H)	Yes
Doubling (1, 2, 4, 8 ...) until a Y, then binary search between the last N and that Y	about 2 log2(A)	No

Binary search is only as good as its upper bound. Without a tight one you need a safe one, and a safe one can be huge. Section 5 shows what that costs on real traffic.

---

4. Why it's correct

The search. At every step, everything below `lo` is known N, and `best` is either nothing yet or `hi + 1`, checked and passed. The range halves each step, so it ends. At the end, `best` was checked and passed, and `best - 1` was checked and failed (unless `best` is the start). A test checks both are in the log.

The check must be monotone, or binary search returns a wrong number and nothing looks broken. For the simulator here (one first-come, first-served queue, each request to the first free machine), adding a machine never makes any request later:

1. Treat n machines as n + 1 where one is never free. Sort each side's "free at" times.
2. At the start, the side with the real extra machine is no later in every position.
3. Each request takes the earliest free time on each side, so it starts no later, and the slot it fills is no later. Sorting keeps the order position by position: if x <= y entry by entry, then sorted(x) <= sorted(y) entry by entry.

So every request's TTFT is no larger, and so is any percentile of it. Tests check this request by request on random traces.

This holds for this simulator. A real load test is noisy and routes differently, so its row can come out N Y N Y near the answer. Section 5 shows what that does to the search.

---

5. On real traffic: the Azure LLM inference trace

Inputs, and where each comes from:

Input	Value	Source
Traffic	Two traces of about an hour from Azure LLM inference services, coding (8,819 requests) and conversation (19,366), replayed as recorded	Azure LLM Inference Dataset 2023, CC-BY
Target	p99 of (TTFT / the same request's TTFT on an idle DGX-A100) at most 6, for A100 and H100 alike	Splitwise, Table VI (Patel et al., ISCA 2024)
One machine	Llama-70B on one DGX machine, 8 GPUs	Splitwise
Idle TTFT	A straight line in prompt tokens, through Splitwise's Table IV medians (see below)	**Our fit**

The idle TTFT line. Splitwise reports that TTFT "grows almost linearly as the prompt size increases", and its Table IV gives median TTFT without batching: coding 185 ms (A100) and 95 ms (H100), conversation 155 ms and 84 ms. The traces' median prompts are 1,469 and 1,020 tokens. Through those two points:

- A100: (185 - 155) / (1469 - 1020) = 0.0668 ms per token, plus 155 - 1020 x 0.0668 = 86.8 ms fixed
- H100: (95 - 84) / 449 = 0.0245 ms per token, plus 59.0 ms fixed

Two points always fit a line exactly, so this is an estimate, not a validated model. Section 5 shows how much a line 25% off would change the answer.

Results (from `demo/`, reproduce with section 8):

Trace	GPU	Fewest machines	GPUs	Checks: try 1, 2, 3	Checks: binary over 1 to requests	Checks: doubling
Coding	A100	11	88	11	13	8
Coding	H100	5	40	5	13	6
Conversation	A100	2	16	2	14	2
Conversation	H100	1	8	1	14	1

What the real trace shows:

- **Size for the bursts, not the average.** Spread evenly, the coding traffic would keep 0.57 A100 machines busy. It needs 11, because it comes in bursts. Its busiest clock second had 67 requests (18:31:26), against an average of 2.57 a second, but one second can't decide a p99 that lets 88 requests be slow. The 10-second bursts do: the work arriving in the busiest 10 seconds needs about 9.4 machines just to keep up.
- **A safe upper bound can make binary search worse than trying every n.** With the number of requests as the bound, binary search took 13 to 14 checks, more than trying 1, 2, 3 in every row.
- **Doubling isn't free either.** For small answers, trying in order is hard to beat (coding on H100: 5 checks in order, 6 doubling). It pays off as the answer grows (coding on A100: 8 against 11).

What-if runs (from `demo/FewestGpus.Experiments`, coding trace, A100 unless noted):

Question	Result
The same prompts arriving at random (Poisson, same average rate, 20 seeds)	2 machines instead of 11 (H100: 1 instead of 5). The bursts cost 9 machines
Work in the busiest window, as machines	1 s: 15.8, 10 s: 9.4, 60 s: 2.5. The 10-second window is the useful estimate
Without the busiest 60 seconds (723 requests)	Still 11
200 hours rebuilt from random 10-second blocks of the real one	11 in 58.5%, 7 in 27%, 6 in 6.5%, 12 in 6%. At 11 machines, 6% of those hours miss the target; at 12, none
One machine down at 11	p99 slowdown 6.73, a miss. Average use at 11 is 5.2%
Every fixed 5-minute block must meet the target	12 (H100: 6). At 11, the worst block has p99 7.7x
A noisy check, each request's prefill times its own lognormal factor (sigma 0.3), 1,000 searches	15.1% answer 10, too few. Majority of 3 per check: 6.6%, at 24 load tests. Only the boundary re-checked as a majority of 3: 7.1%, at 14.6. With sigma 0.1, always 11
Noise shared by the whole run (every request in one load test runs hot or cold together)	sigma 0.1: only 42.8% answer 11, spread 8 to 15. sigma 0.3: 15.5%, spread 4 to 21. Re-checking the boundary barely helps
Stopping a failing load test as soon as it has failed	Failing checks are decided in 3 to 14 minutes; passing ones need the whole 57 minutes. The search drops from 7.6 hours to 3.5
The idle line 25% off in slope and fixed part	9 to 13

Target and percentile matter more than anything else:

Target	p90	p95	p99	p99.9
2x	7	9	13	16
3x	6	8	12	13
4x	6	7	12	13
6x	5	7	11	12
10x	5	6	9	11

6. Assumptions and limits

	Status
Search finds the fewest n; check counts in section 3	**Tested** (78 tests, incl. thousands of random cases against trying every n)
Simulator is monotone	**Tested** on random traces, argued in section 4
Traffic, target, machine size	**Cited** (section 5)
Idle TTFT per request	**Our fit** to two published medians
Real GPU latency	**Not measured.** Nothing here ran on a GPU
Batching	**Left out.** A machine does one first token at a time; real servers batch, and batching changes prefill time
Network time, decode	**Left out.** Only queueing plus prefill

	Status
Fleet size	**Not claimed.** The traces are samples of two services; the answers are for this model of these samples
What is sized	**The prompt pool, as an estimate.** No decode and a perfect router push the answer down; no batching pushes it up
A real load test as the check	**Noisy and maybe not monotone.** Re-check the answer and the one below it, or take a majority of several runs (section 5)

The part that carries over unchanged is the search: plug in a real load test as the check, and it makes the same calls.

---

7. Tests

Area	What they protect
Search	the section 2 row check by check; edges of the range; no candidate checked twice; at most log2 checks; 4,000 random cases against trying every n; no overflow near `int.MaxValue`; cancellation
Doubling	its path; cost depends on the answer, not the bound; never above the limit; "no answer" when nothing passes
Simulator	small cases you can do on paper; waiting; ties; nearest-rank percentile; monotone per request; more machines than requests changes nothing
Slowdown target	TTFT over idle TTFT; a short prompt stuck behind a long one; only N to Y as machines are added
Trace and line	CSV parsing and bad rows; arrivals relative to the first request; busiest second and its tie-break; the line passes through Splitwise's four medians (by construction, not as validation)
Guards	every input check, each proven by a test that fails when the check is removed; doubling near `int.MaxValue`; cancellation while doubling; exact percentile ranks
What-if	Poisson keeps prompts and rate; peak work per window; a noisy check with no noise matches the plain one; majority of 3; a bad minute fails the window check but hides in the hourly one; the window check is monotone

---

8. Run it

The trace files aren't in this repo. Download `AzureLLMInferenceTrace_code.csv` and `AzureLLMInferenceTrace_conv.csv` from the dataset page.

```
cd dsa/search/fewest-gpus-for-slo
dotnet test
dotnet run --project demo/FewestGpus.Demo -- --trace <path>/AzureLLMInferenceTrace_code.csv --gpu A100
dotnet run --project demo/FewestGpus.Experiments -- --trace <path>/AzureLLMInferenceTrace_code.csv
```

The what-if run takes several minutes; most of it is the noisy searches.

Requires .NET 10 SDK.

---

9. Common mistakes

Mistake	What goes wrong
Not checking the check is monotone	Binary search returns a wrong answer, silently
`(lo + hi) / 2`	Overflows on large ranges; use `lo + (hi - lo) / 2`
A safe but huge upper bound	More checks than trying every n (section 5)
Sizing on average traffic	Bursts break the target
Returning the top of the range when nothing passes	Claims n works when it doesn't
Trusting one run of a noisy load test near the answer	The search can settle on one machine too few
A p99 over the whole hour	A bad five minutes hides inside it

Sources

- Google, *Site Reliability Engineering*, ch. 4: Service Level Objectives
- NVIDIA NIM: LLM benchmarking metrics
- NVIDIA Triton: Perf Analyzer CLI
- Patel et al. (2024), "Splitwise: Efficient generative LLM inference using phase splitting", ISCA: arXiv:2311.18677
- Microsoft Azure: Azure LLM Inference Dataset 2023, CC-BY. Data not included here.